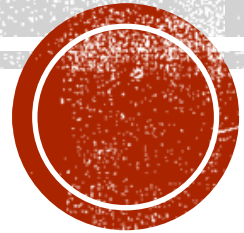
CÓDIGO LIMPIO

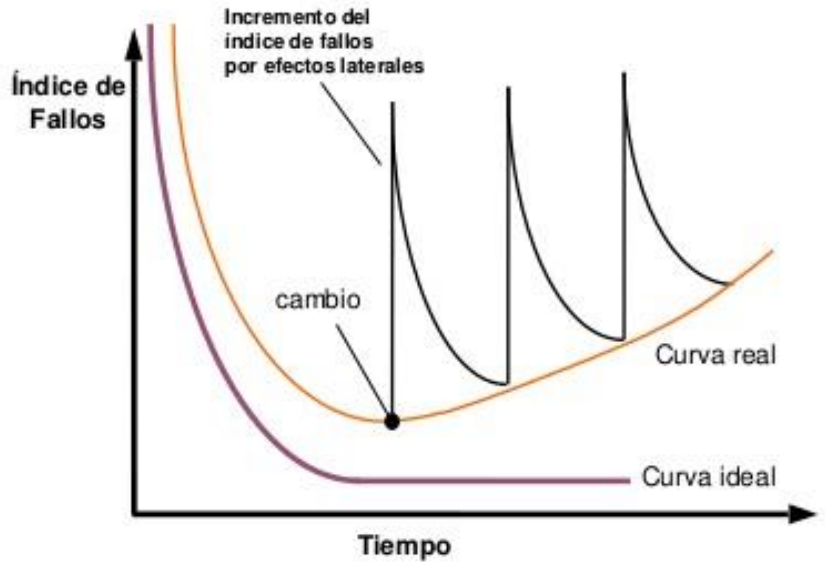
¿Porqué el Software falla?



-basado en el libro "Código Limpio" Robert C Martin-

ANTECEDENTES

La curva de fallos del SOFTWARE



CONCEPTOS DE CÓDIGO LIMPIO

- Me gusta que mi código **sea elegante y eficaz**. La lógica debe ser directa para evitar errores ocultos, las dependencias deben ser mínimas para facilitar el mantenimiento, el procesamiento de errores completo y sujeto a una estrategia articulada, y el rendimiento debe ser optimo para que los usuarios **no tiendan** a estropear el código con optimizaciones sinsentido. El código limpio hace bien una cosa.
- Bjarne Stroustrup – inventor de C++



CONCEPTOS DE CÓDIGO LIMPIO



- El código limpio es siempre y directo. El código limpio se lee como un texto bien escrito. El código limpio **no oculta la intención** del diseñador, sino que muestra **nítidas** abstracciones y líneas directas de control.
- Grady Booch, autor del Análisis orientado a objetos



CONCEPTOS DE CÓDIGO LIMPIO

- El código limpio se puede leer y mejorar por parte de un programador que no sea su autor original. Tienen pruebas de unidad y de aceptación. Tiene nombres con sentido. Ofrece una y no varias formas de hacer algo. Sus dependencias son mínimas se definen de forma explicita y ofrece una API clara y mínima. El código debe ser culto en función del lenguaje, ya que no toda la información necesaria se puede expresar de forma clara en el código.

- "Big" Dave Thomas – Fundador de OTI y estrategia Eclipse

CONCEPTOS DE CÓDIGO LIMPIO

- Podría enumerara todas las cualidades del código limpio, pero hay una principal que engloba a todas ellas. El código limpio siempre parece que ha sido escrito por alguien a quien le importa. No hay nada evidente que hacer para mejorarlo. El autor del código pensó en todos los aspectos posibles y si intentamos imaginar alguna mejora, volvemos al punto de partida y solo nos queda disfrutar de código que alguien a quien le importa realmente nos ha proporcionado.



- Michael Feathers – autor de Working Effectively with Legacy Code

CONCEPTOS DE CÓDIGO LIMPIO

- En los últimos años, comencé y prácticamente terminé con las reglas de código simple de Kent Beck. En orden de prioridad, el código simple:
  - Ejecuta todas las pruebas
  - No contiene duplicados
  - Expresa todos los conceptos de diseño del sistema
  - Minimiza el numero de entidades como clases, métodos, funciones y similares
- De todos ellos, me quedo con **la duplicación**. Cuando algo se repite una y otra vez, es una señal de que teneos una idea que no acabamos de representar correctamente en el código. Intento determinar cual es y después expresar esa idea con mayor claridad. Para mi, la expresividad debe incluir nombres con sentido y estoy dispuesto a cambiar los nombres de las cosas varias veces. **La expresividad** va mas allá de los nombres. También me fijo si un objeto o un método hace mas de una cosa. Si se trata de un objeto, probablemente tenga que dividirse en dos o mas. Si se trata de un método, siempre recurro a la refactorización de extracción de métodos para generar un método que exprese con mayor claridad su cometido y varios métodos secundarios que expliquen como lo hace.



- Ron Jeffries – autor de Extreme Programming



CONCEPTOS DE CÓDIGO LIMPIO

- Sabemos que estamos trabajando con código limpio cuando cada rutina que leemos **resulta ser lo que esperábamos**. Se puede denominar código atractivo cuando el código hace que parezca que el lenguaje se ha creado para el problema en cuestión.

- Ward Cunningham – creador de Extreme Programming

CONCEPTOS DE CÓDIGO LIMPIO

- La regla del Boy Scout
  - No basta con escribir código correctamente. El código debe limpiarse con el tiempo. Los boy scout tienen una sencilla regla que podemos aplicar a nuestra profesión:

"Dejar el campamento más limpio de lo que se ha encontrado"

Si todos entregamos el código mas limpio de lo que lo hemos recibido**, no se corromperá**. No hace falta que la limpieza sea masiva. Cambien el nombre de una variable, divida una función demasiada extensa, elimine elementos duplicados, simplifique una instrucción *if* compuesta.

Robert C Martin – coautor del Manifiesto Ágil



CÓDIGO LIMPIO

```
public List<int[]> getThem() {
    List<int[]> list1 = new ArrayList<int[]>();
    for (int[] x : theList)
        if (x[0] == 4)
            list1.add(x);
    return list1;
}
```

```
public List<Cell> getFlaggedCells() {
    List<Cell> flaggedCells = new ArrayList<Cell>();
    for (Cell cell : gameBoard)
        if (cell.isFlagged())
            flaggedCells.add(cell);
        return flaggedCells;
}
```




GRACIAS.